

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	P. Lohith Krishna	Reg. No.:	192325122
Date:	05/08/2026	Duration:	60 minutes

Code of Conduct

I P. Lohith Krishna 192325122(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Lohith Krishna

Annexure A – Architecture Design Assignment

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
 - ✓ Automate tax calculation based on the latest tax rules and tax slabs.
 - ✓ Reduce manual errors in tax calculation and financial management.
 - ✓ Maintain secure records of users' income, expenses, investments, and tax deductions.
 - ✓ Generate accurate tax reports and financial statements.
 - ✓ Help users create budgets and manage their finances effectively.
 - ✓ Provide tax-saving suggestions and financial planning recommendations.
 - ✓ Send reminders for tax filing deadlines and important financial activities.
 - ✓ Ensure secure user authentication and protection of financial data.
 - ✓ Allow administrators to manage users and update tax rules.

- ✓ Improve efficiency, accuracy, and overall financial decision-making for individuals and small businesses.
- Analyze the functional and non-functional requirements that influence the software architecture.

ANALYSIS OF FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS		
Smart Tax Calculation and Financial Planning Management System		
 FUNCTIONAL REQUIREMENTS These requirements define the specific functions and features the system must provide.		
No.	Functional Requirements	Impact on Architecture
1	User Registration and Login	- Requires separate modules/services for user management, tax calculation, reports, notifications, etc. - Needs robust database design to store financial records securely. - Requires APIs for notifications, payments, and tax rule updates. - Demands clear separation of concerns for maintainability.
2	Profile Management	
3	Income Management	
4	Expense Management	
5	Investment Management	
6	Tax Deduction Management	
7	Automatic Tax Calculation	
8	Budget Planning	
9	Financial Planning and Recommendations	
10	Tax Report Generation	
11	Financial Report Generation	
12	Report Download (PDF/Excel)	
13	Search and Filter Financial Records	
14	Email/SMS Notifications and Reminders	
15	Online Tax Payment (Optional)	
16	User Management (Admin)	
17	Tax Rule Management (Admin)	
18	Database Backup and Recovery	
 Summary: Functional requirements define what the system should do. They drive the structure of modules, data flow, and interactions in the architecture.		
 NON-FUNCTIONAL REQUIREMENTS These requirements define the quality, performance, and constraints of the system.		
No.	Non-Functional Requirements	Impact on Architecture
1	Performance (Fast response time, quick calculations)	Requires efficient code, optimized queries, caching mechanisms.
2	Security (Authentication, authorization, data encryption)	Needs secure authentication, role-based access control, encryption, audit logs.
3	Availability (24x7 availability, minimal downtime)	Requires reliable servers, load balancing, and failover mechanisms.
4	Scalability (Support increasing users and data)	Requires scalable architecture, database optimization, and cloud support.
5	Reliability (Accurate calculations, fault tolerance, backup)	Needs proper error handling, backup, and recovery mechanisms.
6	Maintainability (Easy updates, modular design)	Requires modular design, clean code, and documentation.
7	Usability (User-friendly interface, easy navigation)	Requires responsive UI/UX design and easy workflows.
8	Compatibility (Cross-browser, cross-device)	Requires standard technologies and responsive design.
9	Data Integrity (Accurate and consistent data)	Requires validation, transactions, and constraints.
10	Compliance (Tax laws, data privacy regulations)	Requires compliance with tax laws, data protection and privacy standards.
 Summary: Non-functional requirements define how well the system should work. They influence technology choices, infrastructure, security, and overall architectural design.		

- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

1. Scalability

- Supports a growing number of users and financial records.
- Allows easy addition of new features and services.
- Supports cloud deployment and database expansion.
- Maintains performance even with increased workload.

2. Maintainability

- Uses a modular architecture for easy maintenance.
- Simplifies software updates and bug fixes.
- Allows easy modification of tax rules and business logic.

- Improves code readability and reusability.
-

3. Reliability

- Ensures accurate tax calculations and financial reports.
 - Provides automatic data backup and recovery.
 - Minimizes system failures through proper error handling.
 - Maintains consistent performance under normal operating conditions.
-

4. Availability

- Provides 24×7 access to users.
 - Minimizes downtime during maintenance.
 - Supports disaster recovery and failover mechanisms.
 - Ensures continuous access to financial data and reports.
-

5. Performance

- Responds to user requests within a few seconds.
 - Performs fast tax calculations and report generation.
 - Optimizes database queries for efficient data retrieval.
 - Supports multiple users simultaneously without performance degradation.
-

6. Security

- Implements secure user authentication and authorization.
- Encrypts passwords and sensitive financial data.
- Uses HTTPS for secure communication.
- Provides role-based access control for different users.
- Maintains audit logs to track user activities.
- Protects against unauthorized access, data breaches, and cyber threats.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).

Selected Architectural Style: Layered (Three-Tier) Architecture

The **Layered (Three-Tier) Architecture** is the most suitable architectural style for the **Smart Tax Calculation and Financial Planning Management System** because it separates the system into independent layers, making it easier to develop, maintain, secure, and scale.

Architecture Layers

1. Presentation Layer (User Interface)

- Provides the user interface for taxpayers, business owners, accountants, and administrators.
- Handles user interactions through web or mobile applications.
- Displays dashboards, reports, and notifications.

2. Business Logic Layer (Application Layer)

- Processes user requests and business rules.
- Performs tax calculations based on tax slabs.
- Manages income, expenses, investments, and financial planning.
- Generates reports and handles notifications.

3. Data Layer (Database Layer)

- Stores user information and financial records.
- Maintains tax rules, reports, and transaction history.
- Handles data retrieval, updates, backup, and recovery.

External Services

- Email/SMS Notification Service
- Payment Gateway
- Government Tax Portal APIs
- Cloud Storage/Backup Services (optional)

Advantages of Layered Architecture

- Clear separation of user interface, business logic, and data.
- Easier maintenance and software updates.
- Improved security through controlled access to data.
- Better scalability for future system expansion.
- Simplifies testing and debugging.
- Supports integration with external APIs and services.
- Encourages code reusability and modular development.
- Reduces system complexity and improves reliability.

- Justify why the selected architecture is the most suitable for the given problem.

The **Layered (Three-Tier) Architecture** is the most suitable architecture for the **Smart Tax Calculation and Financial Planning Management System** because it separates the application into **Presentation Layer, Business Logic Layer, and Data Layer**, making the system organized, secure, and easy to manage.

Reasons for Selecting Layered Architecture

1. Separation of Concerns

- Separates the user interface, business logic, and database into different layers.
- Makes the system easier to understand and maintain.

2. Easy Maintenance

- Changes in one layer do not affect the other layers.
- Tax rules and business logic can be updated without modifying the user interface.

3. Scalability

- Supports an increasing number of users and financial records.
- New modules and features can be added easily.

4. Security

- Protects sensitive financial data through secure authentication and authorization.
- Restricts direct access to the database.

5. Performance

- Optimizes data processing and report generation.
- Reduces unnecessary communication between components.

6. Reusability

- Business logic can be reused across web and mobile applications.
- Common services are shared among different modules.

7. Reliability

- Supports backup, recovery, and error handling.
- Ensures consistent and accurate tax calculations.

8. Easy Integration

- Easily integrates with payment gateways, email/SMS services, and government tax portals.
- Supports future API-based integrations.

9. Testing and Debugging

- Each layer can be tested independently.
- Makes debugging faster and easier.

3. Draw the Software Architecture Diagram

Step 1: Users (Actors)

Users interact with the system through a web browser or mobile application.

- Individual Taxpayer
- Small Business Owner
- Chartered Accountant (CA)
- Financial Advisor
- System Administrator

↓ (HTTPS / Internet)

Step 2: Presentation Layer (User Interface)

This layer provides the interface for users to access the system.

- Login & Registration
- Dashboard
- Income & Expense Forms
- Tax Calculation Screen
- Financial Reports
- Notifications

↓ (REST API / HTTPS)

Step 3: Business Logic Layer (Application Layer)

This layer processes user requests and performs all business operations.

- Authentication & Authorization
- User Management
- Income & Expense Management
- Investment Management
- Tax Deduction Module
- Tax Calculation Engine
- Budget Planning
- Financial Planning
- Report Generation
- Notification Service
- Admin Management
- Tax Rule Management

↓ (SQL / JDBC / ORM)

Step 4: Data Layer (Database)

Stores and manages all application data.

- User Database
- Income Records
- Expense Records

- Investment Records
- Tax Rules Database
- Financial Reports
- Audit Logs
- Backup Database



Step 5: External Systems & APIs

The system communicates with external services when required.

- Government Tax Portal API
- Email Service API
- SMS Gateway API
- Payment Gateway API
- Cloud Backup Service

4. Explain Components and Their Interactions

A. Purpose and Responsibilities of Each Architectural Component

1. Presentation Layer (User Interface)

Purpose:

- Provides an interface for users to interact with the system.

Responsibilities:

- User registration and login.
- Display dashboards and reports.
- Accept income, expense, and investment details.
- Show tax calculation results.
- Display notifications and reminders.

2. Business Logic Layer (Application Layer)

Purpose:

- Processes user requests and executes business rules.

Responsibilities:

- Authenticate users.
- Validate user input.
- Calculate taxes based on tax rules.
- Manage income, expenses, and investments.
- Generate financial and tax reports.
- Manage budgets and financial planning.

- Send notifications.
 - Apply updated tax regulations.
-

3. Data Layer (Database)

Purpose:

- Stores and manages all application data securely.

Responsibilities:

- Store user information.
 - Store income and expense records.
 - Store tax rules and deductions.
 - Maintain financial reports.
 - Store audit logs.
 - Perform data backup and recovery.
-

4. External Systems / APIs

Purpose:

- Integrate the application with external services.

Responsibilities:

- Government Tax Portal API – Tax filing and validation.
 - Email/SMS API – Send notifications and reminders.
 - Payment Gateway API – Process online tax payments.
 - Cloud Storage API – Backup and restore data.
-

B. Communication and Data Exchange

1. The user sends requests through the **Web/Mobile Interface**.
 2. The **Presentation Layer** forwards the request to the **Business Logic Layer** using **REST APIs (HTTPS)**.
 3. The **Business Logic Layer** validates the request and processes business rules.
 4. If data is required, the Business Logic Layer communicates with the **Database** using **SQL/JDBC/ORM**.
 5. The **Database** returns the requested information to the Business Logic Layer.
 6. The Business Logic Layer sends the processed results back to the Presentation Layer.
 7. The Presentation Layer displays the results to the user.
 8. When required, the Business Logic Layer communicates with **External APIs** (Government Portal, Email/SMS, Payment Gateway) using **REST APIs**.
-

C. Overall Workflow of the System

Step 1

User registers or logs into the system.

↓

Step 2

User enters income, expense, investment, and tax deduction details.



Step 3

The Business Logic Layer validates the information.



Step 4

The Tax Calculation Engine calculates tax using the latest tax rules.



Step 5

The calculated data is stored in the database.



Step 6

The system generates tax reports and financial summaries.



Step 7

Users can view or download reports.



Step 8

The Notification Module sends reminders for tax filing and payments.



Step 9

Administrators update tax rules and manage user accounts when required.



Step 10

External APIs process online payments, send notifications, and support government tax filing.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

1. Scalability

- The layered architecture supports an increasing number of users and financial records.
- New modules and features can be added without affecting existing components.
- The database and application servers can be scaled independently.
- Suitable for cloud deployment and future expansion.

2. Security

- User authentication and role-based authorization protect system access.
- Passwords and sensitive financial data are encrypted.
- HTTPS ensures secure communication between users and the system.
- Audit logs track user activities for security monitoring.
- Only authorized administrators can update tax rules and manage users.

3. Performance

- Business logic is separated from the user interface for faster processing.
- Optimized database queries improve data retrieval speed.
- Automatic tax calculations and report generation are completed quickly.
- Supports multiple users simultaneously with minimal response time.

4. Reliability

- Automatic database backup prevents data loss.
- Error handling and recovery mechanisms improve system stability.
- Accurate tax calculations ensure dependable results.
- The system continues to operate consistently under normal workloads.

5. Maintainability

- The system is divided into independent layers, making updates easier.
- Tax rules can be modified without changing the user interface.
- Bug fixes and software enhancements can be implemented efficiently.
- Modular design simplifies testing and debugging.

6. Availability

- The system is available **24×7** for users.
- Regular backups and recovery mechanisms reduce downtime.
- Cloud hosting and redundant servers improve service availability.
- Scheduled maintenance has minimal impact on users.

6. Justify Architectural Decisions

A. Rationale Behind the Architectural Choice

The **Layered (Three-Tier) Architecture** was selected because it separates the system into the **Presentation Layer**, **Business Logic Layer**, and **Data Layer**. This separation makes the application easier to develop, test, maintain, and secure.

Reasons for Choosing Layered Architecture

1. Separates the user interface, business logic, and database.
2. Improves code organization and modularity.
3. Simplifies maintenance and software updates.
4. Provides better security by restricting direct database access.
5. Supports scalability as the number of users grows.
6. Makes testing and debugging easier.
7. Allows easy integration with external APIs and services.
8. Reduces system complexity and improves reliability.

B. Trade-offs Considered While Selecting the Architecture

1. Communication between layers may slightly increase response time.
2. Developing multiple layers requires more initial effort.
3. More components increase the complexity of deployment.
4. Small applications may not require a layered design.
5. However, the advantages of modularity, security, and maintainability outweigh these limitations for this project.

C. Support for Future Enhancements, System Integration, and Long-Term Maintainability

Future Enhancements

1. Add AI-based tax-saving recommendations.
2. Develop Android and iOS mobile applications.
3. Introduce cloud-based storage and backup.
4. Add multilingual support.
5. Include advanced analytics and dashboards.

System Integration

- 1. Integrate with Government Tax Portal APIs.
- 2. Connect with banking and payment gateway services.
- 3. Integrate Email and SMS notification services.
- 4. Support third-party accounting software through APIs.
- 5. Enable cloud services for backup and synchronization.

Long-Term Maintainability

- 1. Individual layers can be updated independently.
- 2. New features can be added without affecting existing modules.
- 3. Tax rules can be modified easily when regulations change.
- 4. Modular design simplifies debugging and testing.
- 5. Reusable components reduce development time and maintenance costs.
- 6. Supports regular software updates with minimal downtime.

Conclusion

The **Layered (Three-Tier) Architecture** is the most appropriate choice for the **Smart Tax Calculation and Financial Planning Management System** because it provides **high security, scalability, performance, reliability, maintainability, and availability**. It also supports **future enhancements, seamless integration with external systems, and long-term software maintenance**, making it a robust and efficient architecture for the proposed system.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and	Produces a clear, complete, and well-	Diagram is mostly complete with	Diagram includes only	Diagram is incomplete,

Component Design (20%)	structured architecture diagram with all major components, interfaces, and interactions accurately represented.	minor omissions or notation issues.	essential components and lacks sufficient detail.	unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.